Smart Attendance Management System

1. Denny Mathew Cyriac
2. Christy Arun Mathew
3. Justin Joven Malakkaran
4. Theertha Venugopal

Guided by :

Dr. Ranju S Kartha

CONTENTS

Sl.no.	Heading	Page.no.
1	Problem Definition	3
2	Purpose and Need	4
3	Project Objective	6
4	Literature Surveys	8
5	Architecture diagram	11
6	Proposed Method	12
7	Requirements	15
8	Module Division and Scheduling	17
9	Conclusion	19
10	References	20

PROBLEM DEFINITION

Current attendance methods are fundamentally flawed, as they cannot prevent **proxy attendance** or track students who leave mid-session. There is a critical need for a **persistence-tracking solution** that identifies "bunking" in real-time through interval scanning, which manual methods and static biometric scanners fail to capture. As educational institutions scale, an automated system is essential to **reclaim instructional time**, improve student accountability, and provide administrators with accurate, tamper-proof data for safety and performance analytics.

PURPOSE AND NEED

The primary purpose of this system is to **digitize and automate** classroom administration by repurposing existing CCTV infrastructure into an intelligent monitoring tool. By integrating **teacher-authenticated session starts** with continuous facial recognition, the project aims to eliminate the time-consuming burden of manual roll calls. It seeks to provide a **high-fidelity digital audit trail** that records not just entry, but the actual duration of student presence, ensuring that academic credit is based on genuine classroom engagement.

PROJECT OBJECTIVE

Project Objective

The core objective is to engineer a **fully autonomous, vision-based attendance ecosystem** that transforms standard classroom CCTV into an intelligent administrative tool. The system aims to replace obsolete manual logging with a **"Presence-over-Time"** model, ensuring that attendance is not just recorded at the door but verified throughout the entire lecture.

PROJECT OBJECTIVE

To achieve this, the project focuses on:

- **Zero-Interaction Logging:** Automating the entire attendance process—from teacher-triggered session activation to student recognition—without requiring physical contact or manual entry.
- **Persistence Verification:** Implementing periodic biometric scans to detect "bunking" or mid-class exits, providing a more granular view of student participation than traditional "check-in" systems.
- **Operational Efficiency:** Eliminating the **10-15% time-wastage** associated with manual roll calls, allowing educators to focus exclusively on instruction while the system handles compliance and reporting in the background.
- **Data Integrity:** Building a tamper-proof digital database that integrates with school timetables to ensure accurate, real-time reporting for parents and administrators.

LITERATURE SURVEYS

LITERATURE SURVEYS

1. Recognition & Authentication

Full Reference: An, X., et al. "Partial FC: Training 10 Million Identities on a Single GPU." *CVPR*, 2022. (Integrated into **InsightFace**).

- **Core Contribution:** Optimizes the "ArcFace" logic to handle massive datasets with high efficiency. It is the current gold standard for minimizing "inter-class similarity" (preventing the system from confusing two similar-looking students).
- **Drawback:** Implementation complexity. Setting up the Partial FC loss function in a basic Python script is significantly harder than using a pre-trained model; it's designed for large-scale training rather than simple inference.

Full Reference: Li, M., & Dang, Q. "gcsamTfaceNet: A lightweight face recognition algorithm fusing Transformer and CNN." *ITM Web of Conferences*, 2025 (Published late 2024).

- **Core Contribution:** Combines CNNs (for local features) with Transformers (for global dependencies) to better handle facial occlusions (like masks or hands) and varied lighting in classrooms.
- **Drawback:** Higher computational overhead. While "lightweight," a pure Python implementation on a standard CPU may struggle with real-time processing of 60 simultaneous students compared to traditional CNNs.

LITERATURE SURVEYS

2. Tracking & Persistence Analysis

Full Reference: Aharon, N., et al. "BoT-SORT: Robust Associations Multi-Pedestrian Tracking." *arXiv preprint*, 2023.

- **Core Contribution:** An evolution of DeepSORT. It improves "Track ID" persistence by integrating camera motion compensation. This is vital if your classroom CCTV rotates or if students move seats during a session.
- **Drawback:** Sensitivity to low frame rates. If your CCTV stream lags or the Python processing loop is slow, BoT-SORT can lose the "Track ID," resulting in a student being marked "absent" because the system thinks they are a "new" person.

Full Reference: Li, D. S. "FRFormer: FaceReconstructor Transformer for low-resolution high-noise face images." *Journal of Computer Science & Information Technology*, 2024.

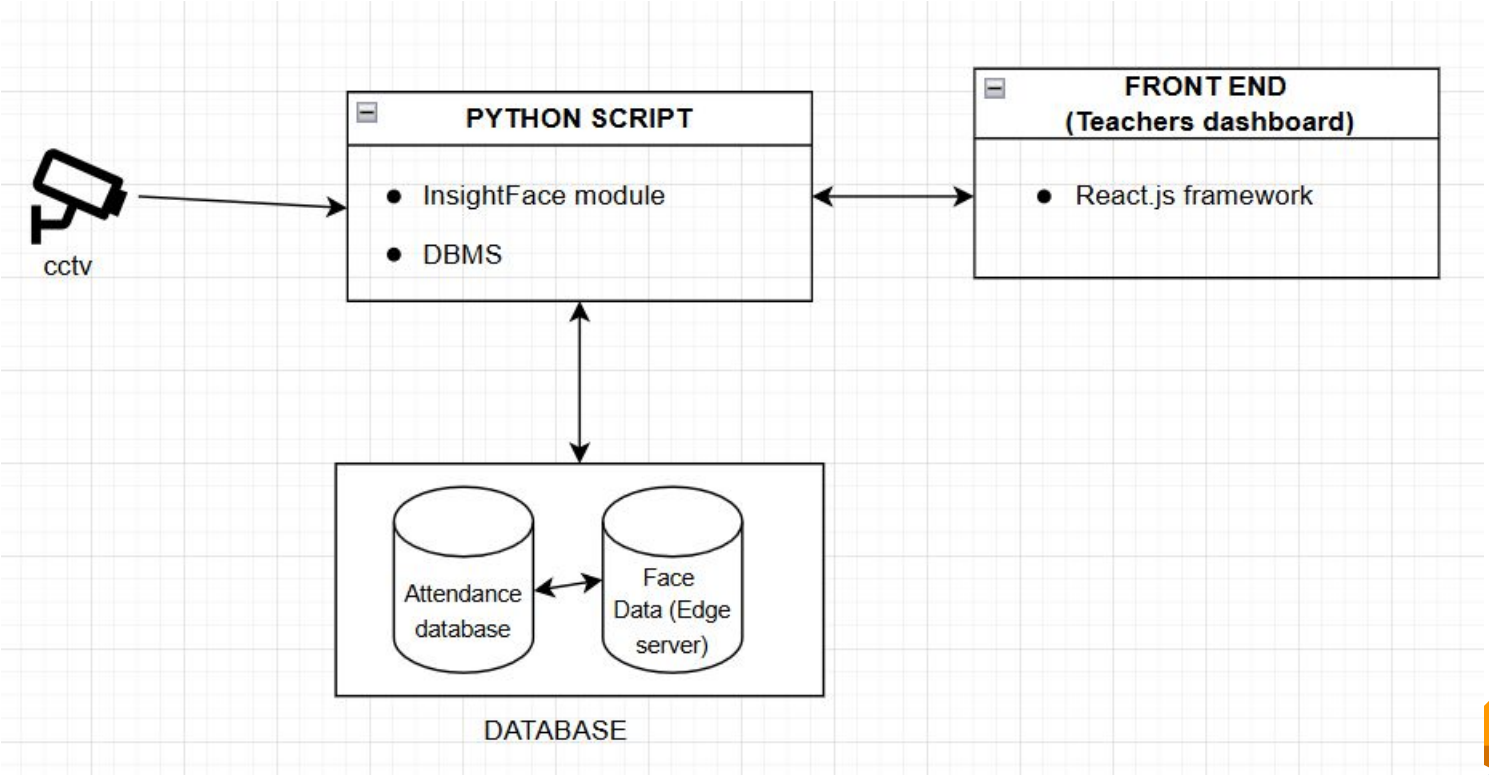
- **Core Contribution:** Specifically designed for low-resolution, noisy CCTV footage. It reconstructs facial features from grainy images to improve recognition accuracy in the back rows of a classroom.
- **Drawback:** Latency. The "reconstruction" phase adds a delay to the pipeline, which might interfere with your goal of recursive scanning every 5–10 minutes if you are processing a high-resolution 4K stream.

3. Automated "Bunk" & Event Logic

Full Reference: Wang, et al. "Face recognition with age as a feature using deep transfer learning." *RSIS International*, 2024.

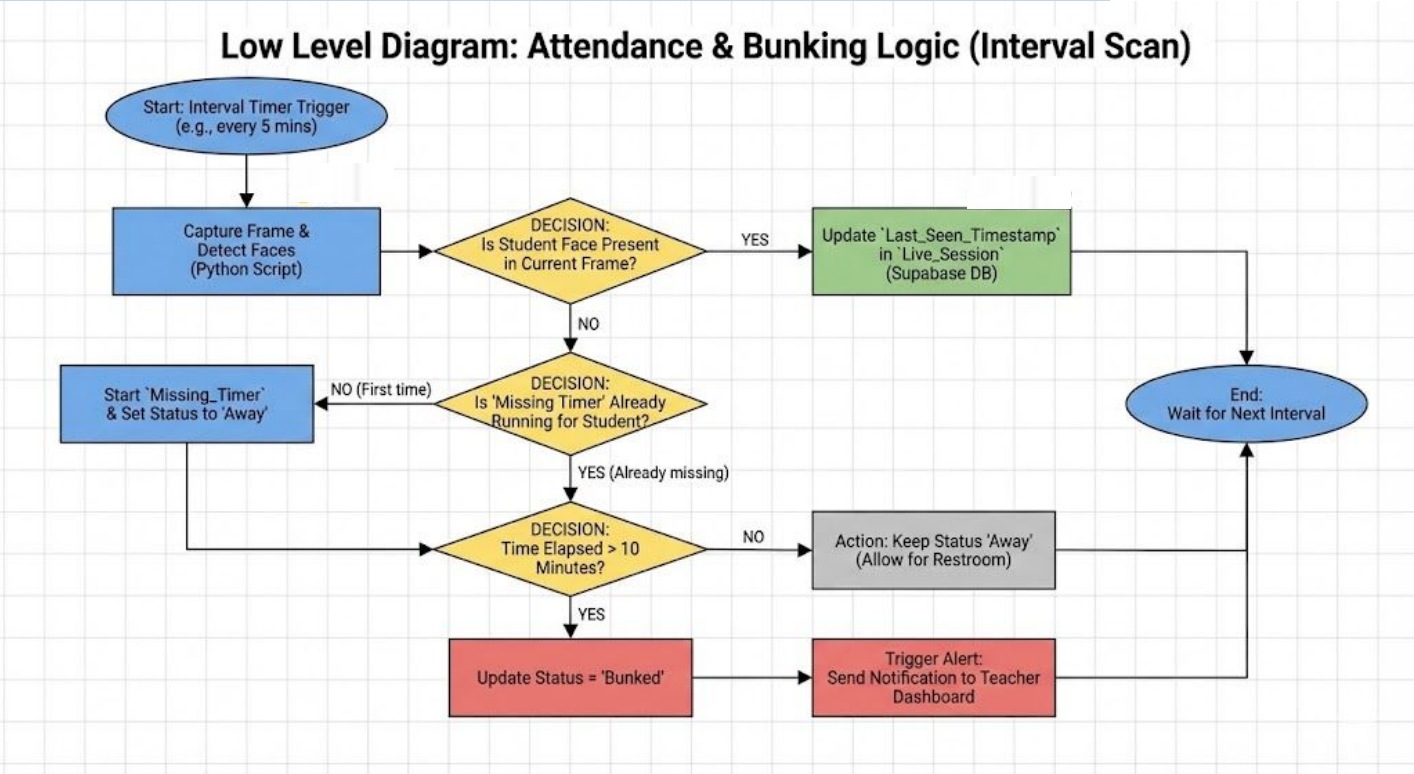
- **Core Contribution:** Focuses on aging and expression variation. This ensures that as students change (haircuts, glasses, or facial growth over a school year), the "Bunk Detection" doesn't falsely flag them due to a failed recognition match.
- **Drawback:** Data Bias. These models often require very diverse training data. Using a simple "one-photo" registration in your Supabase DB might not be enough to leverage the model’s full accuracy.

ARCHITECTURE DIAGRAM

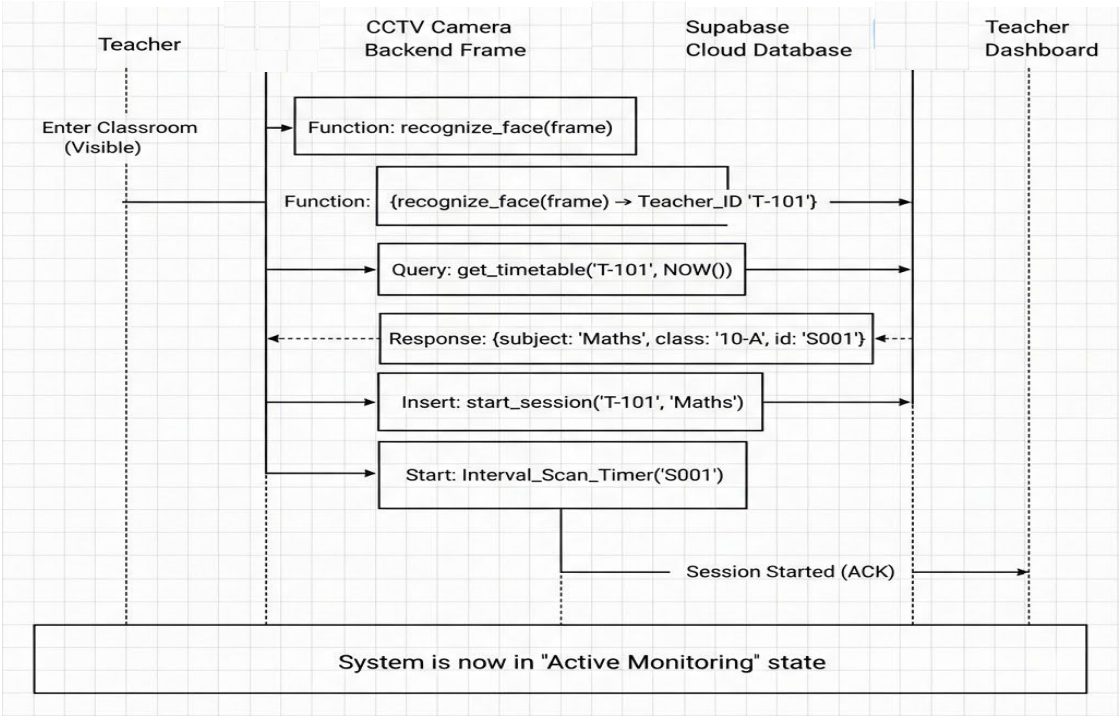


BASIC IMPLEMENTATION

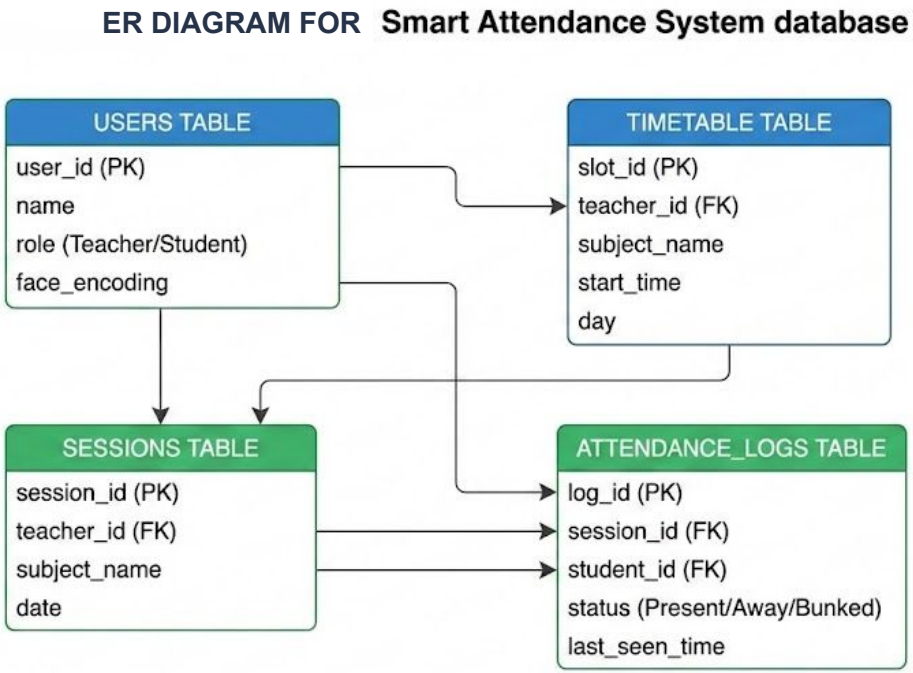
Low level diagrams



Low level diagrams



Low level diagrams



PROPOSED METHOD

System Initialization & Authentication

The first stage focuses on securing the session and ensuring the system knows *what* to record.

- **Teacher Face ID Trigger**: The system remains in a passive state until the camera detects and verifies the authorized teacher's biometric signature.
- **Timetable Integration**: Once authenticated, the system queries the institutional database to fetch the specific subject, class, and duration scheduled for that time slot.
- **Session Binding**: The attendance log is officially "opened" and linked to both the teacher's ID and the subject, preventing manual entry errors or unauthorized logging.

PROPOSED METHOD

Recognition & Tracking Pipeline

This slide covers the computer vision "heavy lifting" that occurs once the class begins.

- **Multi-Face Detection:** The system captures high-resolution frames from the CCTV and utilizes a detection model to locate all students in the frame simultaneously.
- **Feature Extraction & Matching:** Each detected face is converted into a mathematical embedding and compared against the student database using a recognition engine.
- **Crowd Density Mapping:** To handle occlusions (students sitting behind others), the system generates a density map to ensure every occupant in the room is accounted for, even if their face is partially blocked.
- **Identity Deduplication:** The software ensures that each unique student is counted only once per session, regardless of movement.

PROPOSED METHOD

Persistence Monitoring & Bunking Logic

The final stage is what makes your system unique: tracking student presence *over time*.

- **Recursive Interval Scanning:** Rather than a single "roll call," the system performs automated scans at variable intervals (e.g., every 5–10 minutes) throughout the period.
- **Multi-Object Tracking (MOT):** Using algorithms, the system maintains a consistent "track ID" for every student, monitoring their coordinates within the classroom.
- **Bunking Detection Logic:**
  - **The 2-Scan Rule:** If a student’s facial signature is missing for two consecutive interval scans, they are flagged.
  - **Early Exit Alert:** If the student does not return before the session ends, a "Bunk" or "Early Exit" status is automatically pushed to the instructor's dashboard and the central database.
- **Final Log Sync:** At the end of the period, the system calculates the "Total Presence Time" and closes the session.

REQUIREMENTS

Hardware Requirements

Since you are using existing infrastructure, the hardware focus is on "Edge" processing (the computer running the code) and the capture device.

- **CCTV Camera / High-Def Webcam:** Minimum 1080p resolution is required to ensure that facial features of students sitting in the back rows are clear enough for encoding.
- **Host Machine (PC/Laptop):** A computer with at least 8GB RAM and a dedicated GPU (like NVIDIA) is recommended to handle real-time frame processing, though a fast CPU can handle interval-based scans.
- **Teacher Interface:** A tablet, smartphone, or desktop computer where the teacher can view the live count and receive "Bunking" alerts.
- **Local Network (Wi-Fi/Ethernet):** A stable connection to push data from the local Python script to your Supabase cloud database.

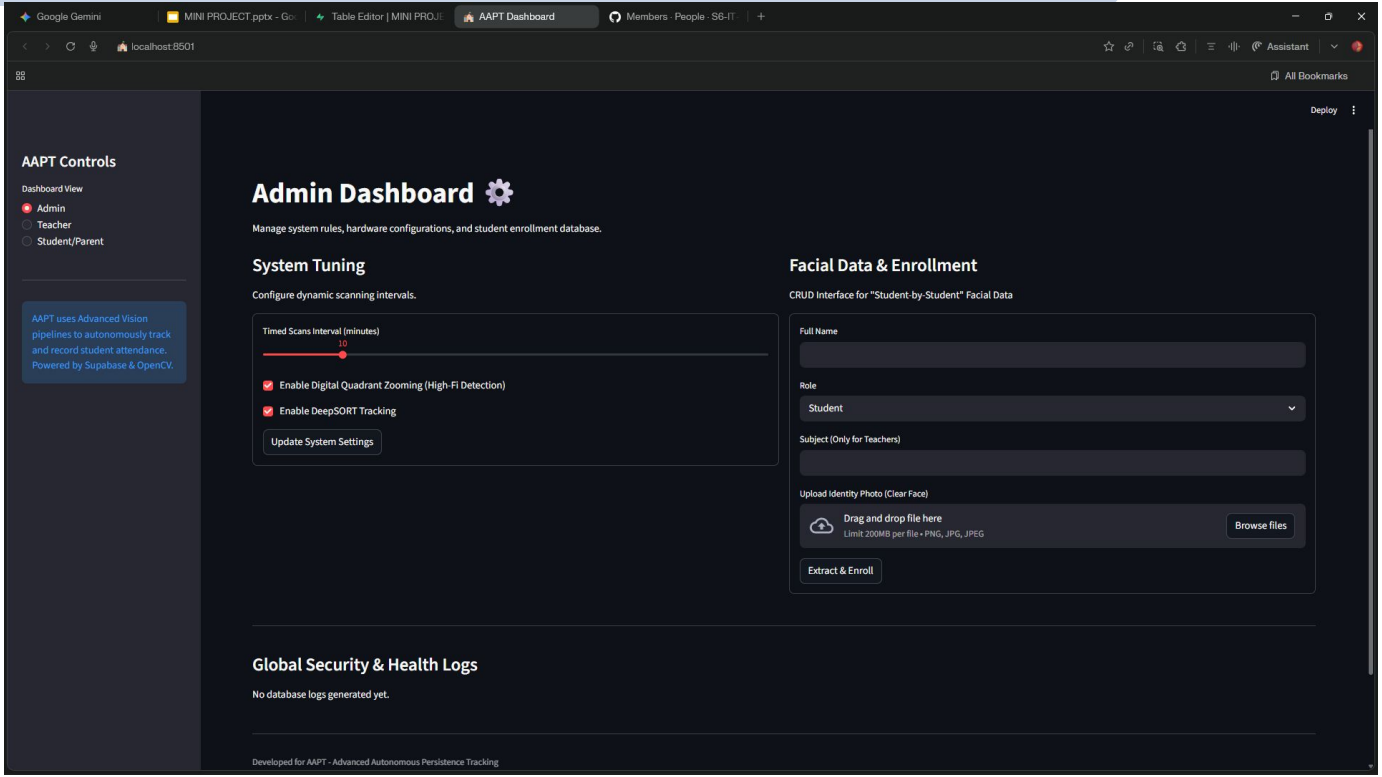
REQUIREMENTS

Software Requirements:

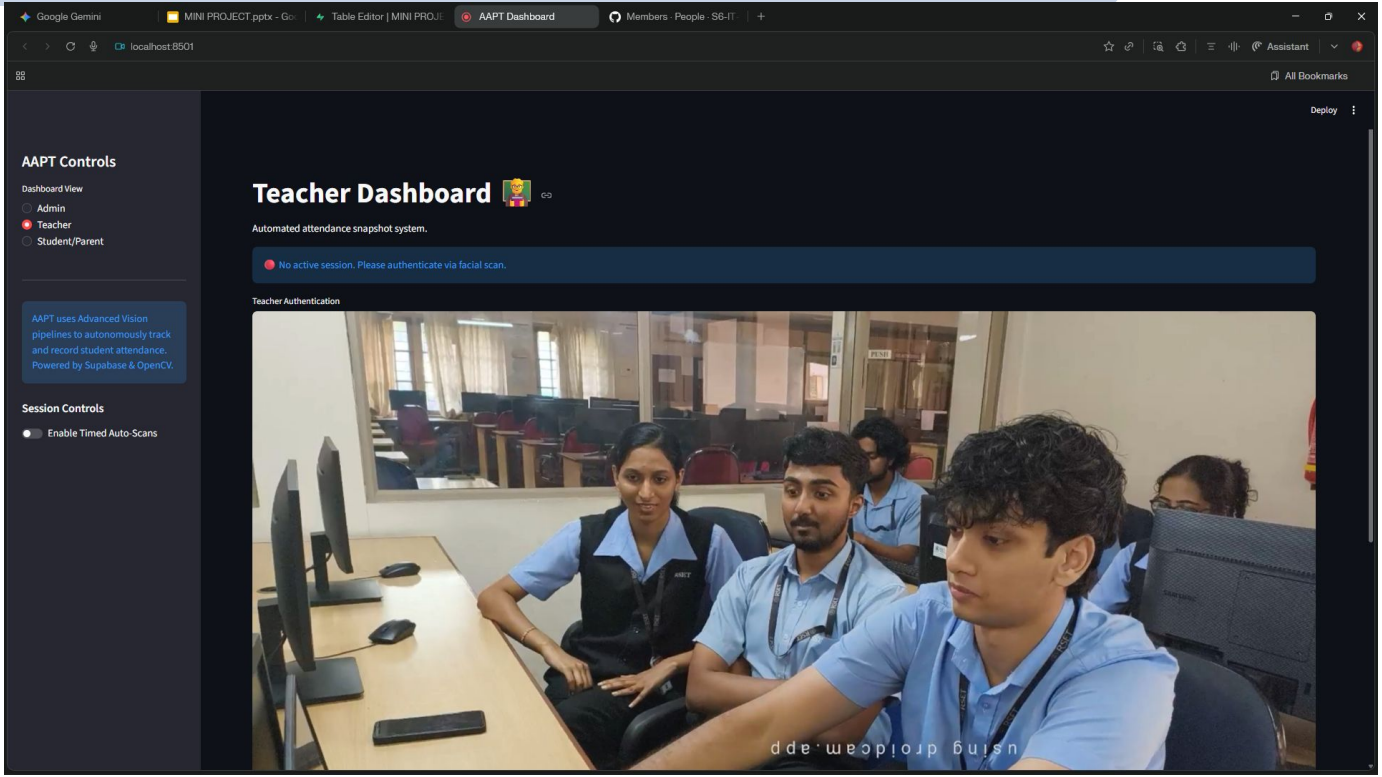
This is the "Brain" of the project, focusing on the libraries that are used in the script and the cloud setup.

- **Operating System:** Windows, macOS, or Linux
- **Programming Language: Python 3.x** (the core language for your face recognition script).
- **Computer Vision Libraries:**
  - **OpenCV:** For video stream handling and image preprocessing.
  - **InsightFace Module:** For generating facial embeddings and matching student identities.
- **Backend & Database (Supabase):** * **PostgreSQL:** To store student profiles, timetable data, and attendance logs.
  - **Supabase Python SDK:** To handle real-time data syncing between the classroom script and the cloud.
- **Development Tools:** * **VS Code / PyCharm:** For script development.
  - **Git:** For version control.

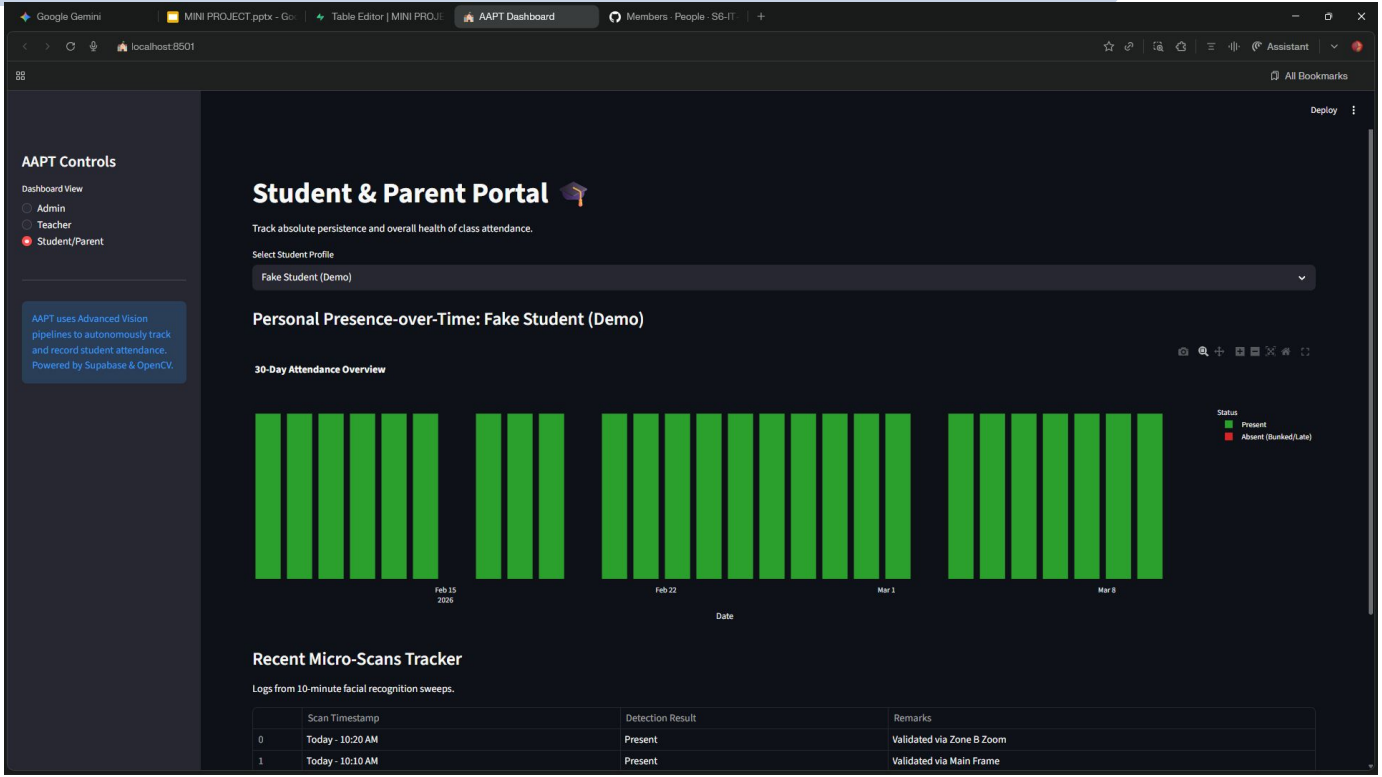
DEMO / SCREENSHOTS



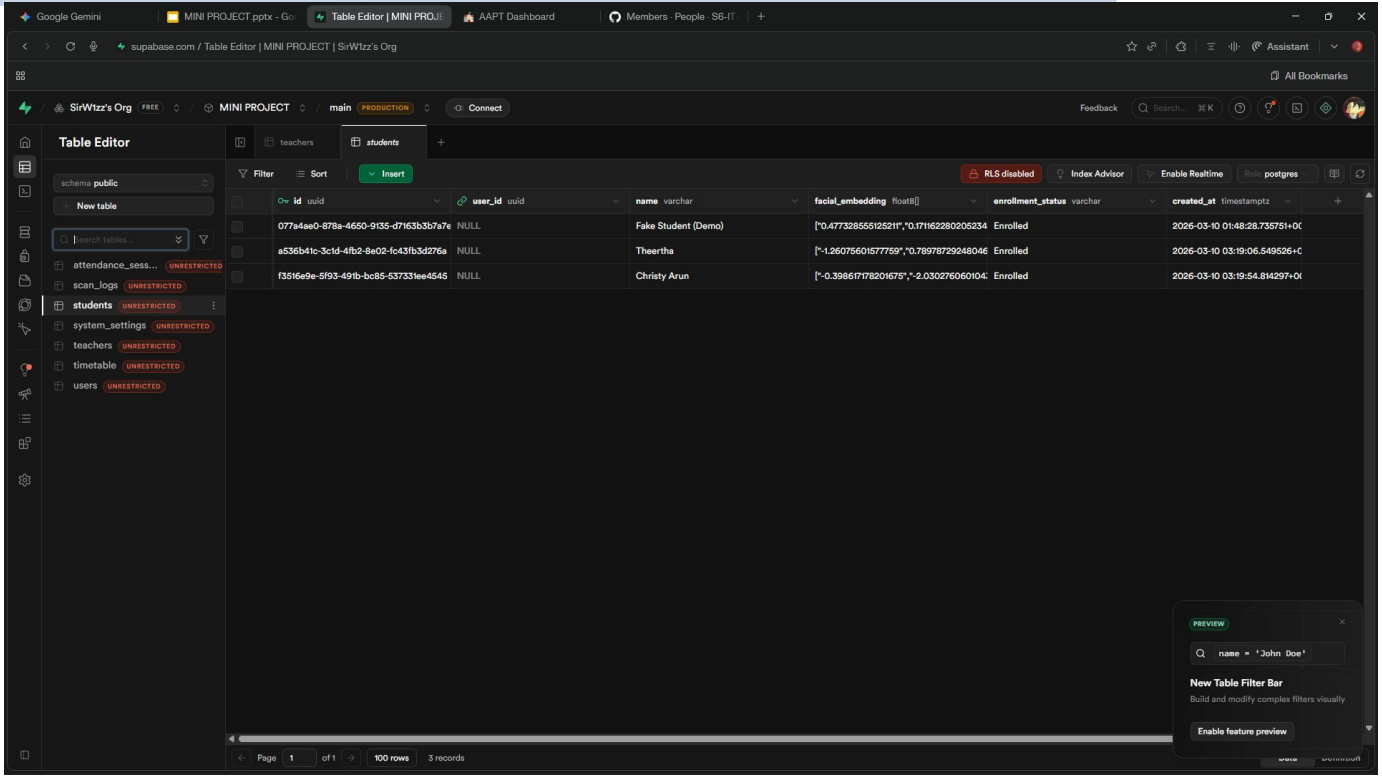
DEMO / SCREENSHOTS



DEMO / SCREENSHOTS



DEMO / SCREENSHOTS



ALGORITHMS USED

Multi-Object Tracking (DeepSORT methodology)
To maintain persistence tracking(a student as they move around the room without needing to run heavy facial recognition on *every single frame*), the system uses a tracking algorithm. It calculates the centroid distance between bounding boxes in consecutive frames.

The "2-Scan Rule" (Temporal Bunk Detection Logic)
The system performs recursive interval micro-scans (e.g., every 10 minutes). Rather than instantly marking a student absent if they drop a pencil and disappear from the camera, the system stores an array of their presence history. The logic dictates that an alert is only triggered if a student registers as `False` (Missing) for two consecutive interval scans.

Digital Quadrant Zooming (High-Fidelity Scanning Algorithm)
Instead of processing a crowded 1080p classroom frame all at once, the system algorithmically splits the frame into 4 equal quadrants (Zones A, B, C, D). It then digitally zooms (resizes) each quadrant to the full processing size before running facial recognition. This drastically improves detection accuracy for students sitting far in the back of the classroom.

ALGORITHMS USED

SORT Methodology)

xes around all faces in the current frame.

center point (centroid) of each bounding box.

center points to the known center points from the previous frame.

hin a set distance threshold (e.g., `< 120 pixels`) of an old center, reassign the same tracking ID.

ned, register it as a newly tracked object.

al Bunk Detection Logic)

de facial recognition micro-scan at set intervals (e.g., 10 minutes).

e of every enrolled student for the current interval as `True` or `False`.

value directly to the student's historical array log.

entries in the array.

arly Exit" alert ONLY if the array registers `[..., False, False]`

High-Fidelity Scanning)

0p classroom frame from the camera.

x into 4 equal quadrants (Zones A, B, C, D).

n) each isolated quadrant back up to the full 1080p resolution.

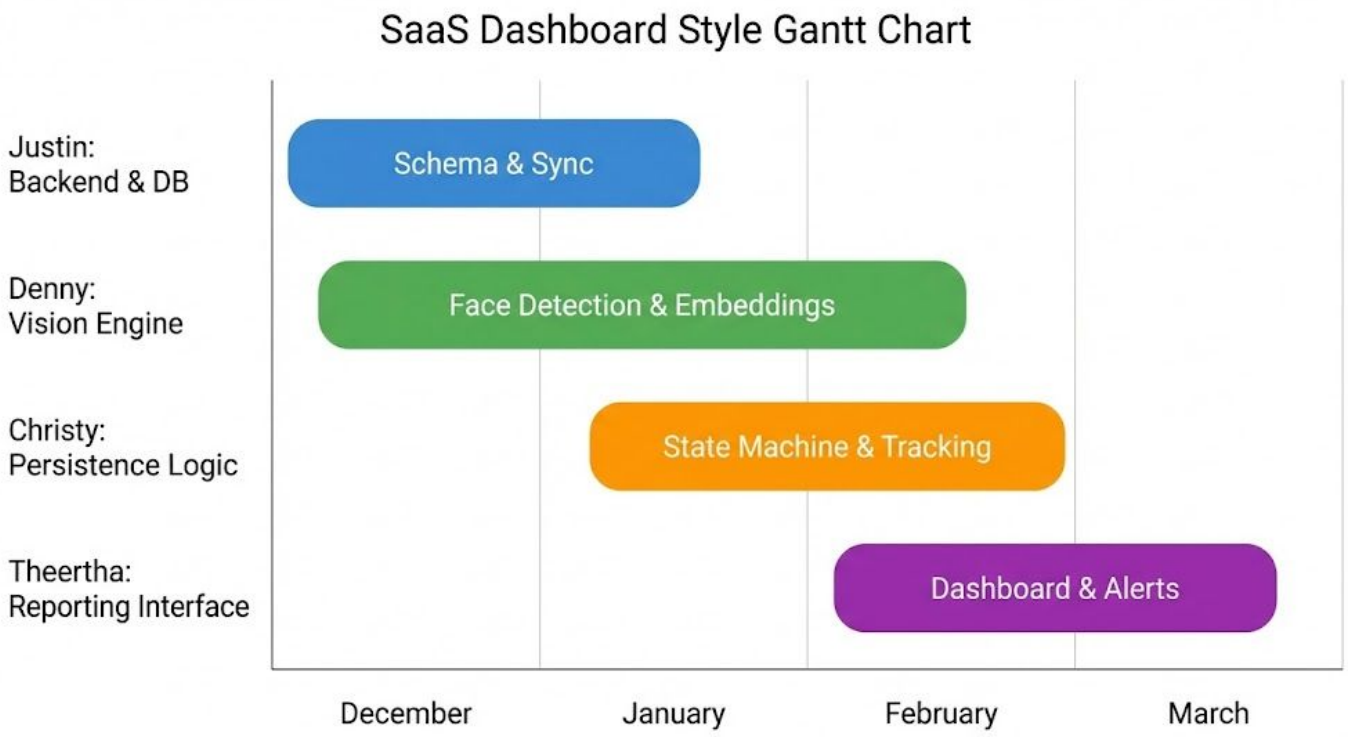
tion model on all 4 zoomed quadrants independently.

cale the coordinates of any detected faces back down to their true positions in the original full frame.

Module Division and Scheduling

Modules	Topic	Content Division	Schedule	Time Period
1	Backend & Database	**Justin:** Supabase environment setup, ER diagram implementation, and timetable-to-subject sync logic.	Weeks 1–3	1 Month
2	Vision Engine	**Denny:** Python-based face detection, encoding generation, and real-time recognition using existing CCTV feeds.	Weeks 3–6	1 Month
3	Persistence Logic	**Christy:** Developing the interval scanning loop, "State Machine" logic for restroom breaks, and bunking detection.	Weeks 6–9	1 Month
4	Reporting Interface	**Theertha:** Building the teacher dashboard, live attendance counter, and automated notification system for exits.	Weeks 9–12	1 Month

GANTT CHART


SaaS Dashboard Style Gantt Chart
Justin:
Backend & DB
Denny:
Vision Engine
Christy:
Persistence Logic
Theertha:
Reporting Interface
Schema & Sync
Face Detection & Embeddings
State Machine & Tracking
Dashboard & Alerts
December
January
February
March

CONCLUSION

The **Automated Attendance and Persistence Tracking (AAPT)** system represents a significant leap from traditional "point-in-time" attendance to a **continuous monitoring model**. By leveraging existing CCTV infrastructure and the power of Python-based computer vision, the system effectively addresses the long-standing challenges of manual errors, proxy attendance, and classroom "bunking."

REFERENCES

1. Core Libraries & Documentation

- **OpenCV (Open Source Computer Vision Library):** Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. https://opencv.org
- **Face Recognition Library:** Geitgey, A. (2017). *Face Recognition: The world's simplest facial recognition api for Python*. https://github.com/ageitgey/face_recognition
- **Supabase Documentation:** Supabase (2024). *The Open Source Firebase Alternative: Database, Auth, and Realtime*. https://supabase.com/docs
- **Dlib Toolkit:** King, D. E. (2009). *Dlib-ml: A Machine Learning Toolkit*. Journal of Machine Learning Research.

2. Key Algorithms & Research Papers

- **DeepSORT (Tracking Logic):** Wojke, N., Bewley, A., & Paulus, D. (2017). *Simple Online and Realtime Tracking with a Deep Association Metric*. IEEE International Conference on Image Processing (ICIP). (This is the basis for your "Bunking" and "Restroom" detection).
- **HOG (Face Detection):** Dalal, N., & Triggs, B. (2005). *Histograms of Oriented Gradients for Human Detection*. IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR).
- **FaceNet (Embeddings):** Schroff, F., Kalenichenko, D., & Philbin, J. (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

REFERENCES

3. Smart Attendance Systems (General Reading)

- **Automatic Attendance:** Visalakshi, S., & Ashish, V. (2019). *Multi-face recognition system using OpenCV and local histogram-based techniques*. International Journal of Scientific Research.
- **Real-Time Monitoring:** Patel, T. (2025). *Face Recognition Attendance System Using Python and Cloud Databases*. GitLab Repository.

4. Technical Web Resources

- **PostgreSQL Official Documentation:** *Schema design and foreign key constraints for relational mapping*. https://www.postgresql.org/docs/
- **Python Language Reference:** Python Software Foundation. *Python Language Reference, version 3.x*. https://docs.python.org/3/